

FUNZIONI DI ORDINE SUPERIORE

↳ funzioni passate come parametro ad altre funzioni

PROBLEMA

Risoluzione di riferimenti non locali
in procedure/funzioni passate come
parametro ad altre funzioni



Regole di visibilità + regole di scoping
non sempre sono sufficienti

ND in particolare nel caso di scoping dinamico
sono necessarie anche delle regole
espressive dette di binding

Esempio:

int $x=4$; int $z=0$; Ambiente valido al momento della definizione

①

int f(int y) { return $x * y$; }

procedura
con
ambiente
non locale

void g(int h(int m)) {

non locale
scoping + binding

int $x=7$;

③ $z = h(3) + x$;

ambiente valido
quando la procedura
viene effettivamente eseguita
mediante il parametro
formale

non locale
(scoping)

:

{ int $x=5$;

② { g(f);

Ambiente valido al momento del
passaggio di f: creazione del
frame con h

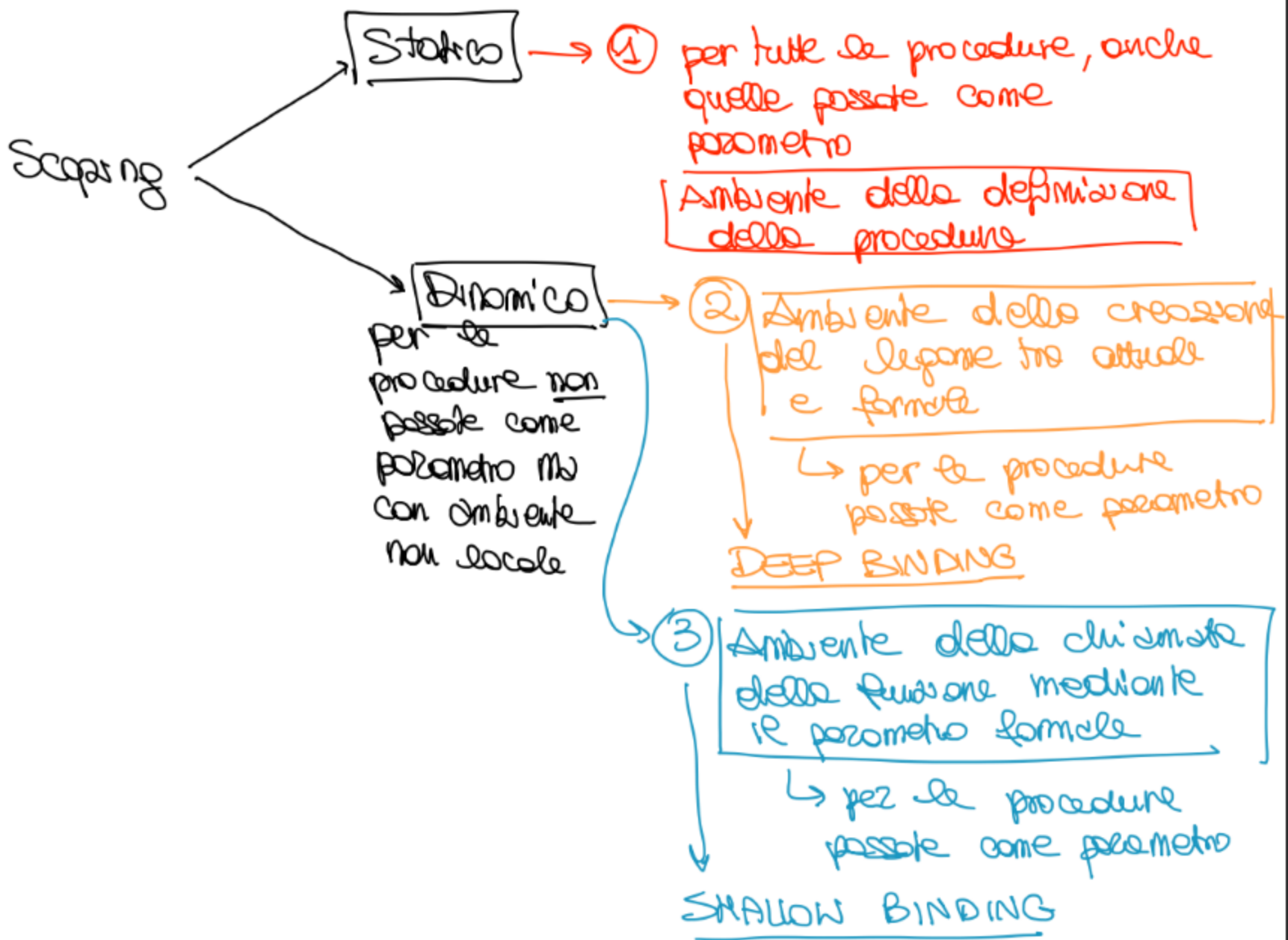
f con ambiente non locale
è passato come parametro
ad un'altra funzione

⇒ scoping non sufficiente

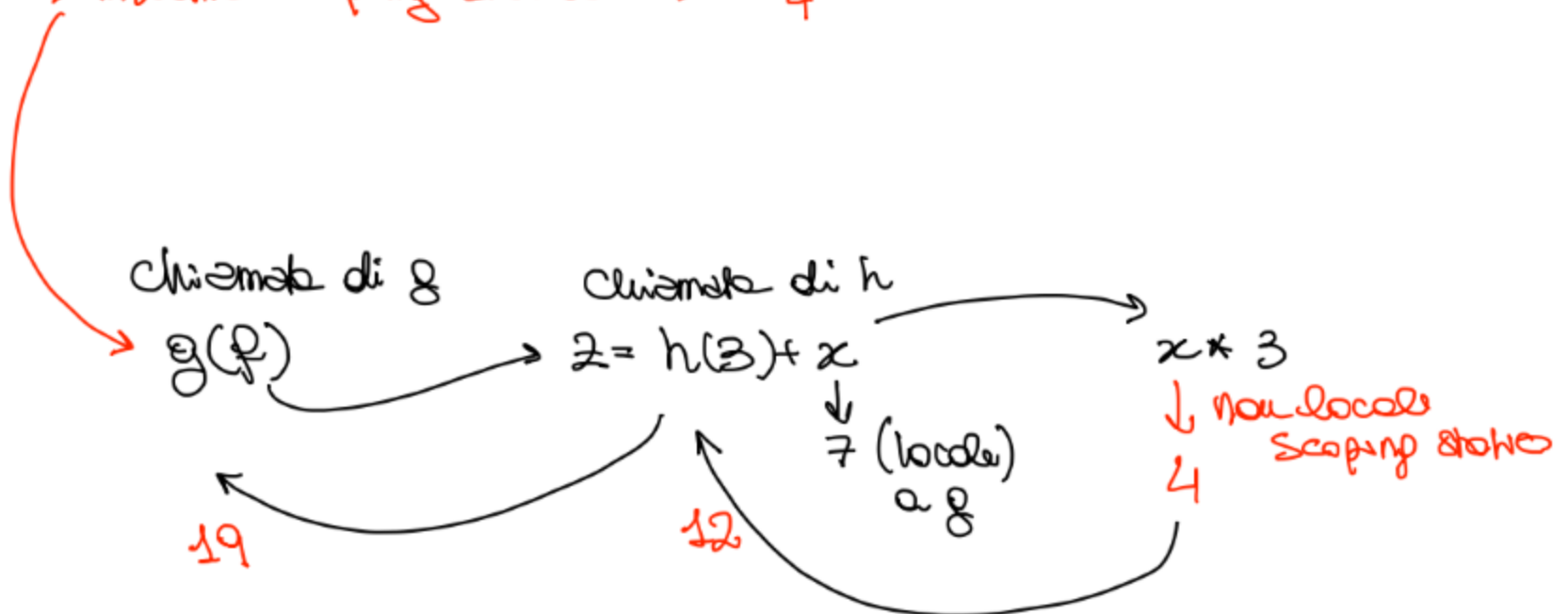
Entro cui sono
ambienti
determinabili
a tempo di
esecuzione

⇓

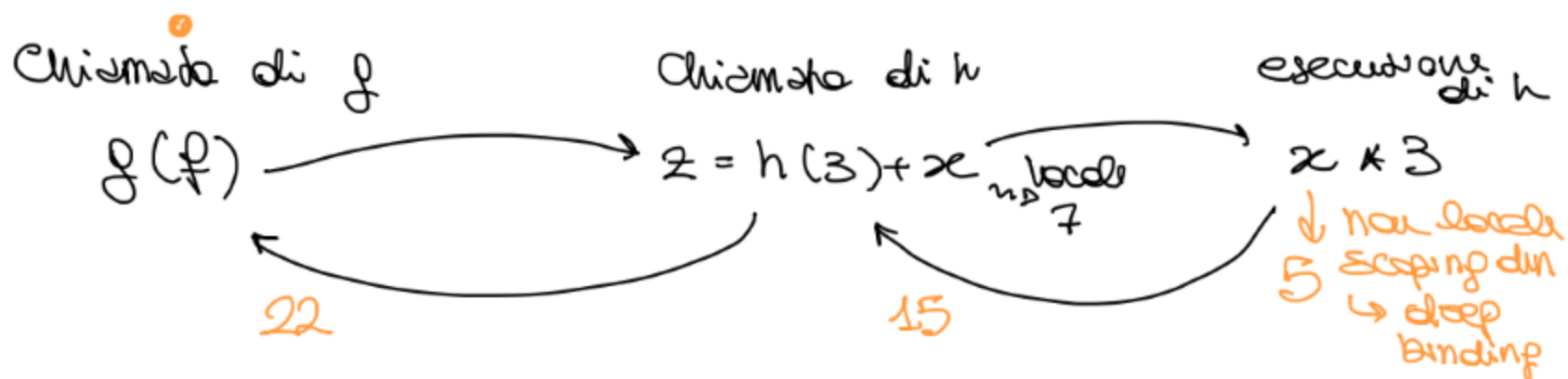
Rilevanti in
caso di
regole di scoping
dinamico



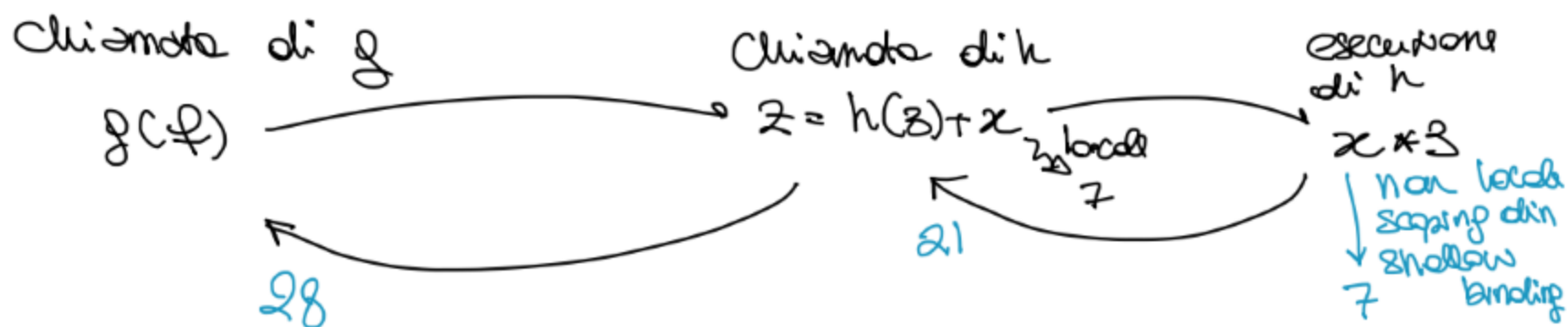
Ambiente scoping statico $x = 4$



Ambiente dinamico con deep binding $x=5$



Ambiente dinamico con shallow binding $x=7$



ESERCIZIO ①

→ Quando si usano le regole di scoping (e se si definiscono se non fatto in esercizi precedenti)

ambiente non locale in procedure

→ Quando si usano le regole di binding

ambiente non locale in procedure
passate come parametro ad altre procedure

→ Definire le regole di binding

scoping statico → Ambiente definizione ①

scoping dinamico + deep binding → Ambiente creazione
lezione tra formule
e attore della
funzione passata ②

scoping dinamico + shallow binding → Ambiente della
chiamata del ③

① Definizione

{ int x=4; }

int p (int a) {
 return x+2;
}

→ locale

non locale

int q (int f (int b)) {

int x=3; ③ chiamata

return f(x)+x;
}

→ locale

int exec () {

int x=2;

return q(p);
}

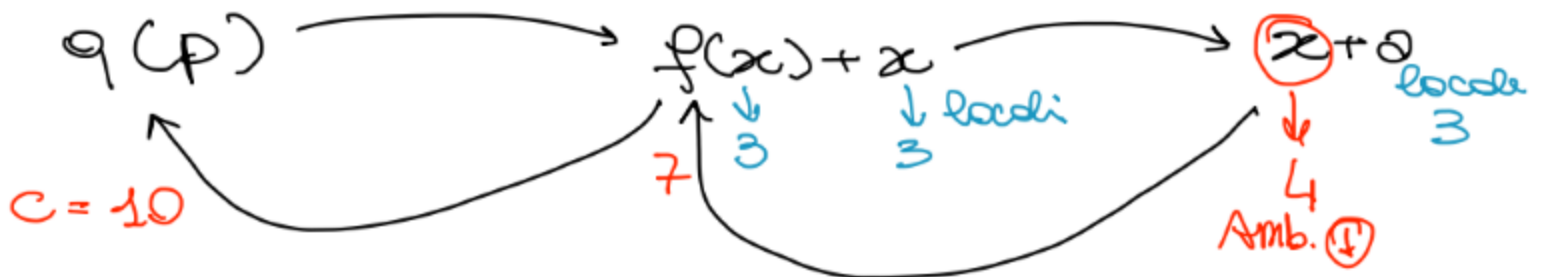
② Creazione del legame

int c = exec();

{

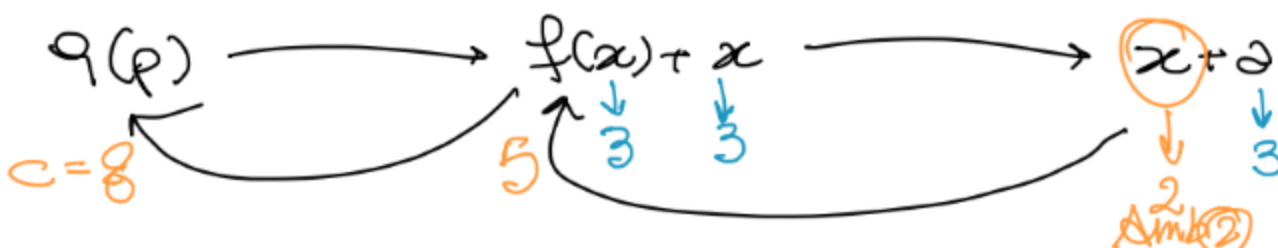
Scoping statico

→ ① Definizione ambiente per x
 x=4



Scoping dinamico + deep binding

→ ② Creazione del legame
 x=2



Scoping dinamico + shallow binding → ③ chiamato $x=3$



Esercizio ②

int x=4; int y=6; ① Chiamato (statico) per tutti

```
int p (int a) {
    return x + y - a;
}
```

non locali → dobbiamo usare regole scoping + bind

```
int q (int f (int b)) {
```

int x=3; ③ chiamato di f (dinamico per f)

```
    return f(x) + y;
}
```

non locale → dobbiamo usare regole di scoping

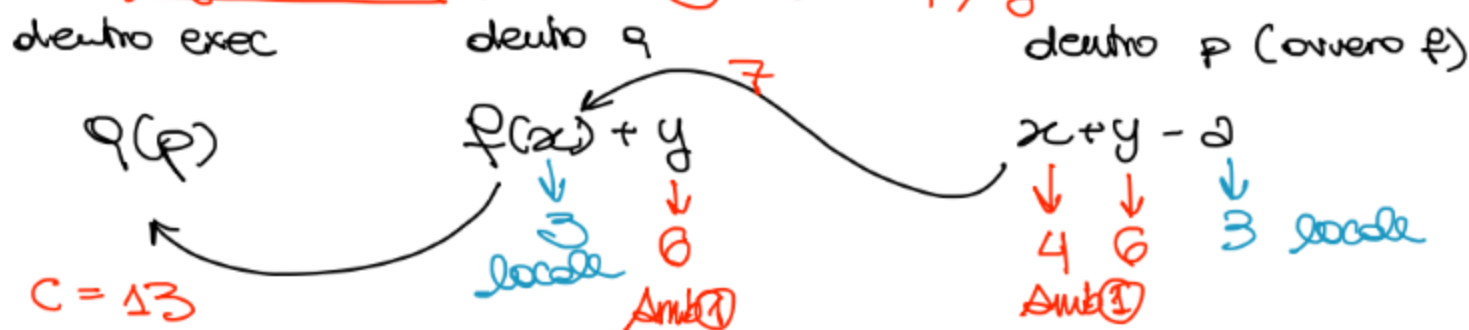
```
int exec() {
```

int y=5; ② creazione legame (dinamico chiamato di q)

```
    return q(p);
}
```

```
int c = exec();
```

Scoping statico: Amb ① $x=4, y=6$



Scoping dinamico + deep binding

per p (ovvero f) $Amb(5)$ scoping + binding
per q $Amb(2)$ scoping

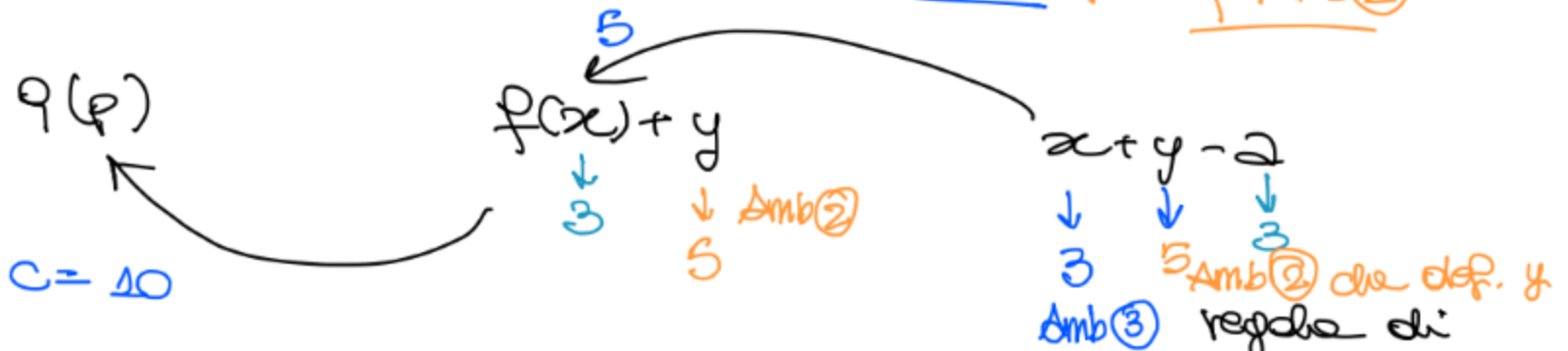


perché $Amb(2)$ non definisce x ma deve risolvere all'ambiente genitore (del chiamante) che è $Amb(4)$

→ regola di visibilità per x

Scoping dinamico + shallow binding

per p (f) $Amb(3)$
per q $Amb(2)$



$Amb(2)$ che def. y
regola di visibilità per y
 $Amb(2)$ precede nello stack $Amb(3)$

→ regola di visibilità per y